Chapter 3: Processing Pipelines

This chapter will show you everything you need to know about spaCy's processing pipeline. You'll learn what goes on under the hood when you process a text, how to write your own components and add them to the pipeline, and how to use custom attributes to add your own metadata to the documents, spans and tokens.

1 Processing pipelines

2 What happens when you call nlp?

3 Inspecting the pipeline

4 Custom pipeline components

5 Use cases for custom components

6 Simple components

7 Complex components

8 Extension attributes

9 Setting extension attributes (1)

10 Setting extension attributes (2)

11 Entities and extensions

12 Components with extensions

13 Scaling and performance

14 Processing streams

15 Processing data with context

16 Selective processing

5 Use cases for custom components

Which of these problems can be solved by custom pipeline components? Choose all that apply!

1. Updating the trained pipelines and improving their predictions
2. Computing your own values based on tokens and their attributes
3. Adding named entities, for example based on a dictionary
4. Implementing support for an additional language

- ○ 1 and 2.
- ○ 1 and 3.
- ○ 1 and 4.
- ◉ 2 and 3.
- ○ 2 and 4.
- ○ 3 and 4.

Submit

That's correct! Custom components are great for adding custom values to documents, tokens and spans, and customizing the `doc.ents`.

✓ Completed Next

- Define the custom component and apply the `matcher` to the `doc`.
- Create a `Span` for each match, assign the label ID for `"ANIMAL"` and overwrite the `doc.ents` with the new spans.
- Add the new component to the pipeline *after* the `"ner"` component.
- Process the text and print the entity text and entity label for the entities in `doc.ents`.

INPUT

```
import spacy
from spacy.language import Language
from spacy.matcher import PhraseMatcher
from spacy.tokens import Span

nlp = spacy.load("en_core_web_sm")
animals = ["Golden Retriever", "cat", "turtle", "Rattus norvegicus"]
animal_patterns = list(nlp.pipe(animals))
print("animal_patterns:", animal_patterns)
matcher = PhraseMatcher(nlp.vocab)
matcher.add("ANIMAL", animal_patterns)

# Define the custom component
@Language.component("animal_component")
def animal_component_function(doc):
    # Apply the matcher to the doc
    matches = matcher(doc)
    # Create a Span for each match and assign the label "ANIMAL"
    spans = [Span(doc, start, end, label="ANIMAL") for match_id, start, end in matches]
    # Overwrite the doc.ents with the matched spans
    doc.ents = spans
    return doc


# Add the component to the pipeline after the "ner" component
nlp.add_pipe("animal_component", after="ner")
print(nlp.pipe_names)

# Process the text and print the text and label for the doc.ents
doc = nlp("I have a cat and a Golden Retriever")
print([(ent.text, ent.label_) for ent in doc.ents])
```

Run code Submit

OUTPUT

```
animal_patterns: [Golden Retriever, cat, turtle, Rattus norvegicus]
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner', 'animal_
component']
[('cat', 'ANIMAL'), ('Golden Retriever', 'ANIMAL')]
```

Show hints Show solution Reset

11 Entities and extensions

In this exercise, you'll combine custom extension attributes with the statistical predictions and create an attribute getter that returns a Wikipedia search URL if the span is a person, organization, or location.

- Complete the `get_wikipedia_url` getter so it only returns the URL if the span's label is in the list of labels.
- Set the `Span` extension `"wikipedia_url"` using the getter `get_wikipedia_url`.
- Iterate over the entities in the `doc` and output their Wikipedia URL.

INPUT

```
import spacy
from spacy.tokens import Span

nlp = spacy.load("en_core_web_sm")


def get_wikipedia_url(span):
    # Get a Wikipedia URL if the span has one of the labels
    if span.label_ in ("PERSON", "ORG", "GPE", "LOCATION"):
        entity_text = span.text.replace(" ", "_")
        return "https://en.wikipedia.org/w/index.php?search=" + entity_text


# Set the Span extension wikipedia_url using the getter get_wikipedia_url
Span.set_extension("wikipedia_url", getter=get_wikipedia_url)

doc = nlp(
    "In over fifty years from his very first recordings right through to his "
    "last album, David Bowie was at the vanguard of contemporary culture."
)
for ent in doc.ents:
    # Print the text and Wikipedia URL of the entity
    print(ent.text, ent._.wikipedia_url)
```

Run code Submit

OUTPUT

```
fifty years None
first None
David Bowie https://en.wikipedia.org/w/index.php?search=David_Bowie
```

Part 2

- Disable the tagger and lemmatizer using the `nlp.select_pipes` method.
- Process the text and print all entities in the `doc`.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")
text = (
    "Chick-fil-A is an American fast food restaurant chain headquartered in "
    "the city of College Park, Georgia, specializing in chicken sandwiches."
)

# Disable the tagger and lemmatizer
with nlp.select_pipes(disable=["tagger", "lemmatizer"]):
    # Process the text
    doc = nlp(text)
    # Print the entities in the doc
    print(doc.ents)
```

Run code Submit

OUTPUT

```
(American, College Park, Georgia)
```

Show hints Show solution Reset